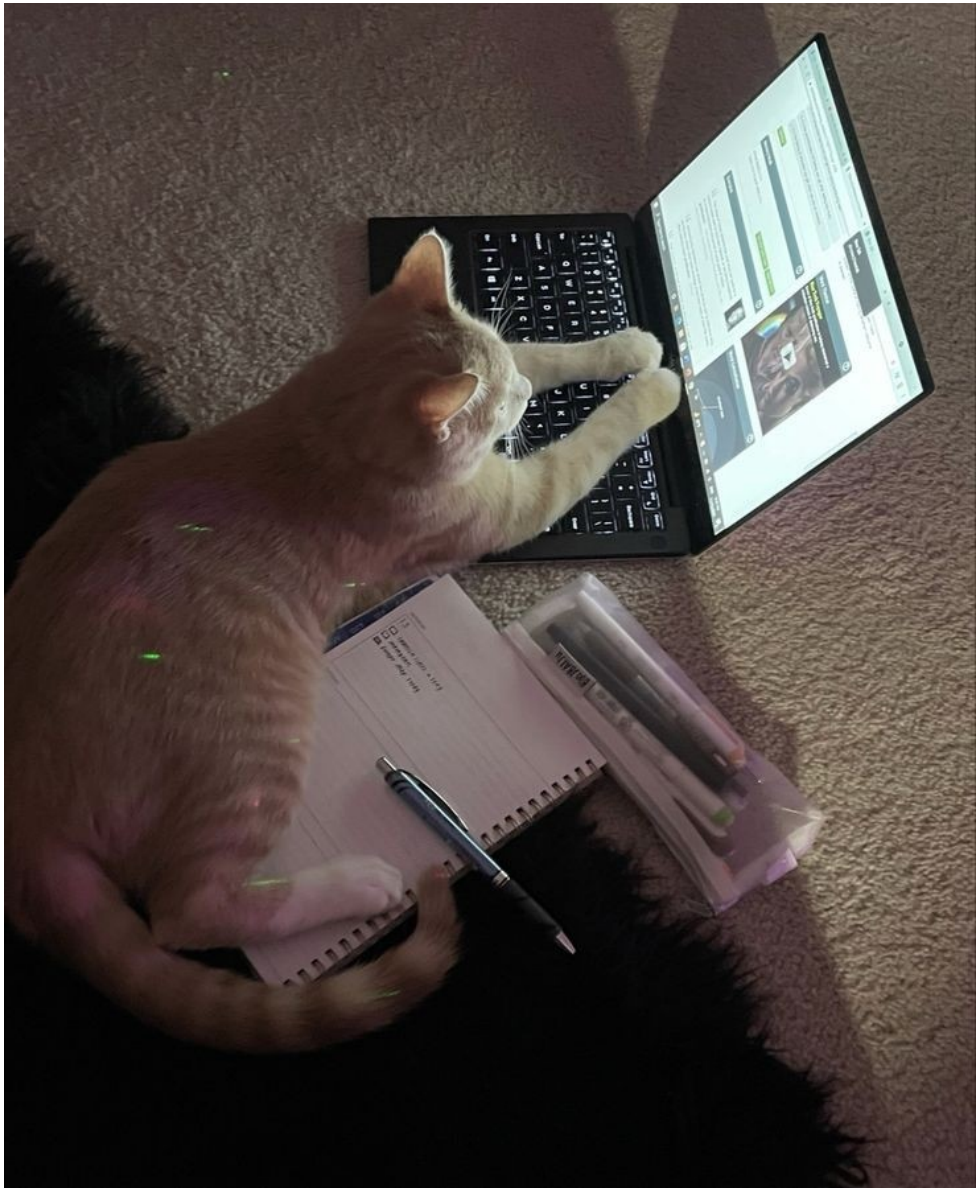


Занятие 1.
Пиксель → цвет → нейрон
Как машина учится видеть

Мастерские МИЦ. 10 сезон
Основы машинного обучения
Занятие 1
Пономаренко Вера

Орг моменты

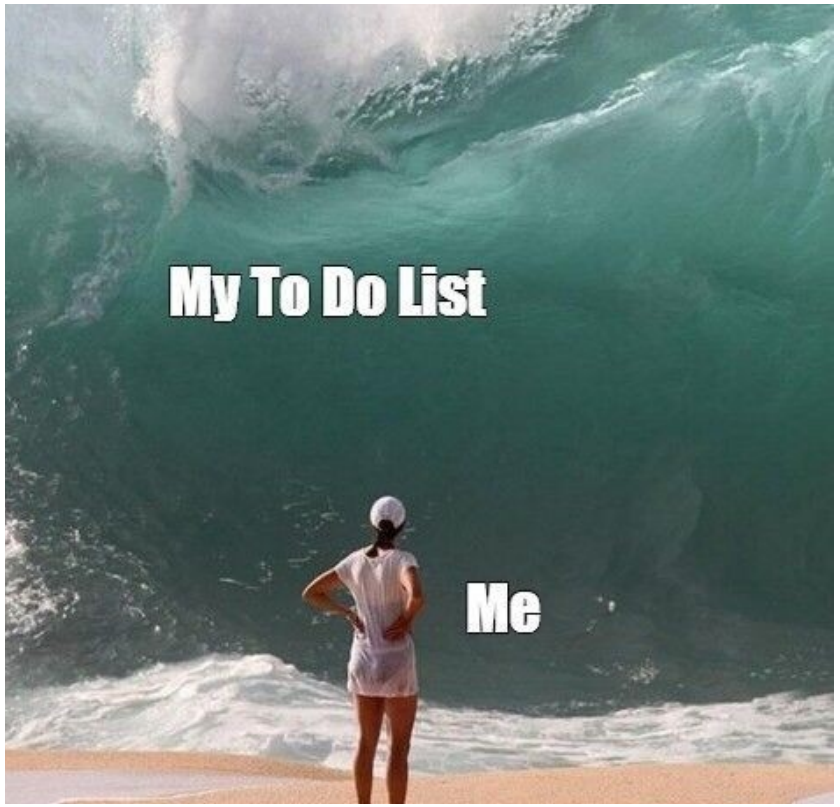
- Группа в тг:

Структура курса:
8 занятий +
8 дз
проект + защита

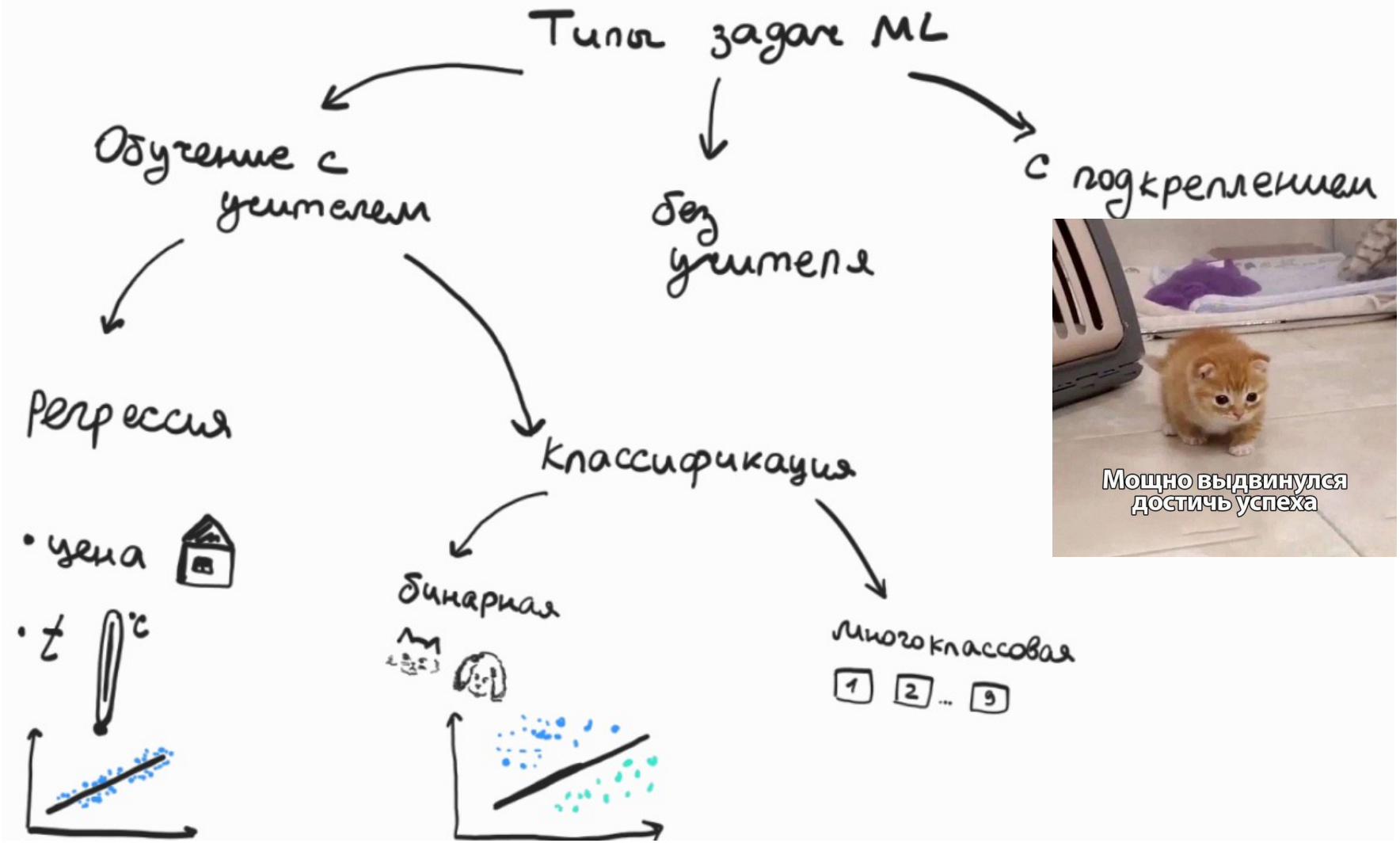


План занятия

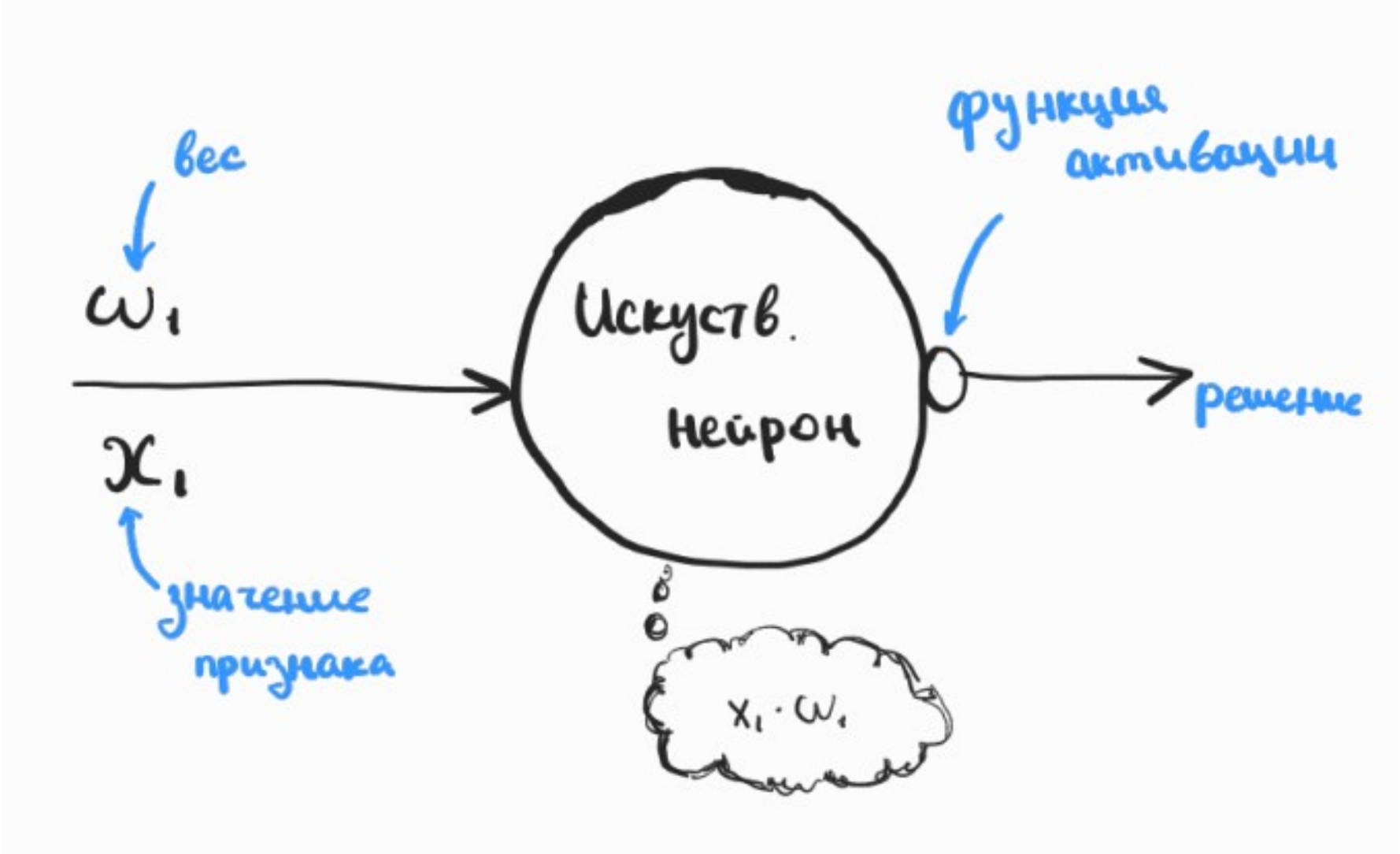
- Основные понятия ML
- Нейрон: веса, смещение, функция активации.
- Изображение как данные
- Цветовые пространства
- Базовые инструменты OpenCV
- Поиск объекта по цвету

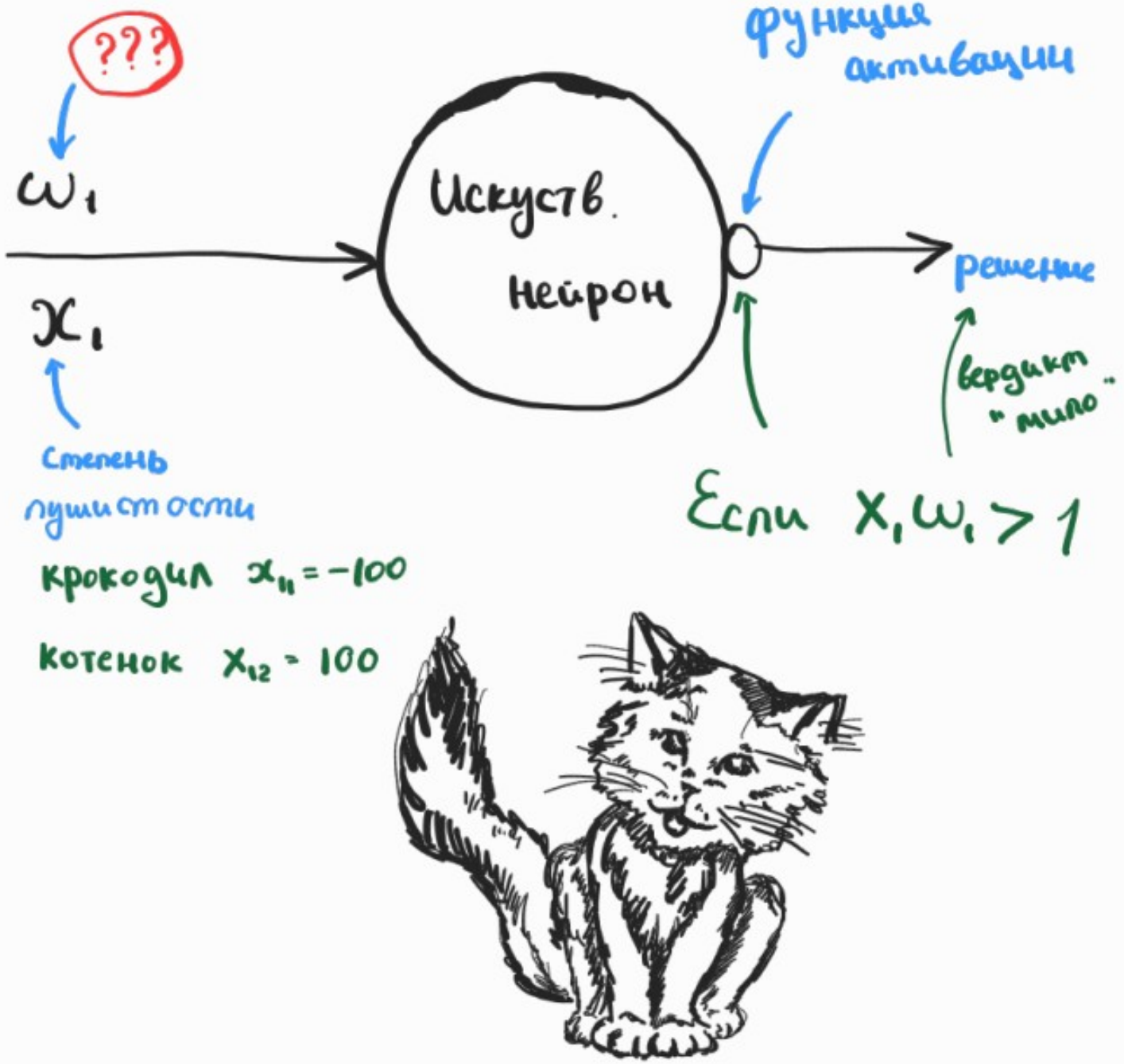


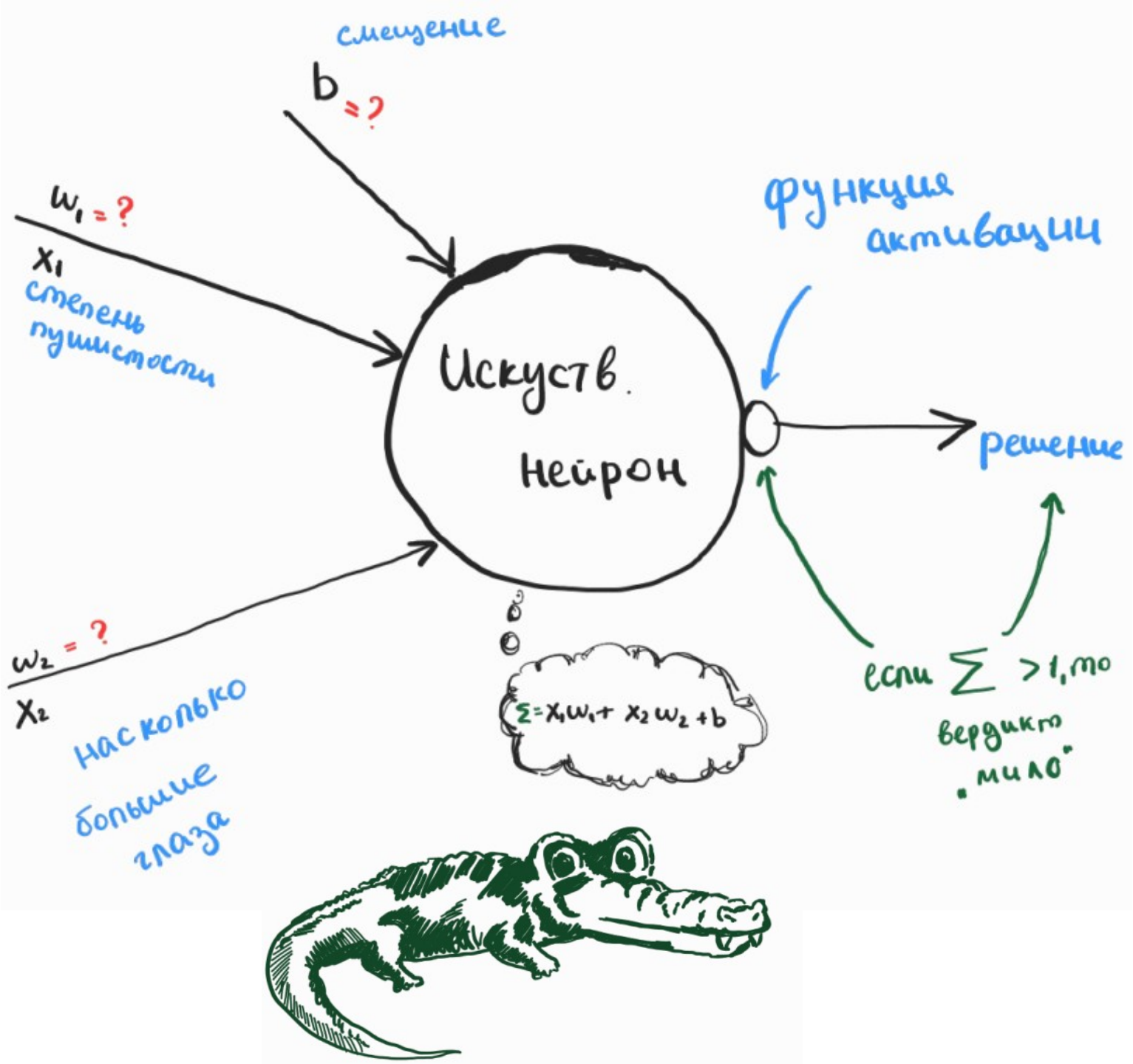
Что такое этот ваш МЛ?..



Искусственный нейрон
Что же учится в машине?




???
ω_1
x_1
Степень лушистости
Искуств. Нейрон
функция активации
решение
вердикт "мило"
Если $x_1 \omega_1 > 1$
крокодил $x_{11} = -100$
котенок $x_{12} = 100$


смещение
b = ?
w1 = ?
x1
степень пушистости
функция активации
Искусств.
Нейрон
решение
w2 = ?
x2
насколько большие глаза
Σ = x1w1 + x2w2 + b
если Σ > 1, то
вердикт "мило"

Нейронная сеть

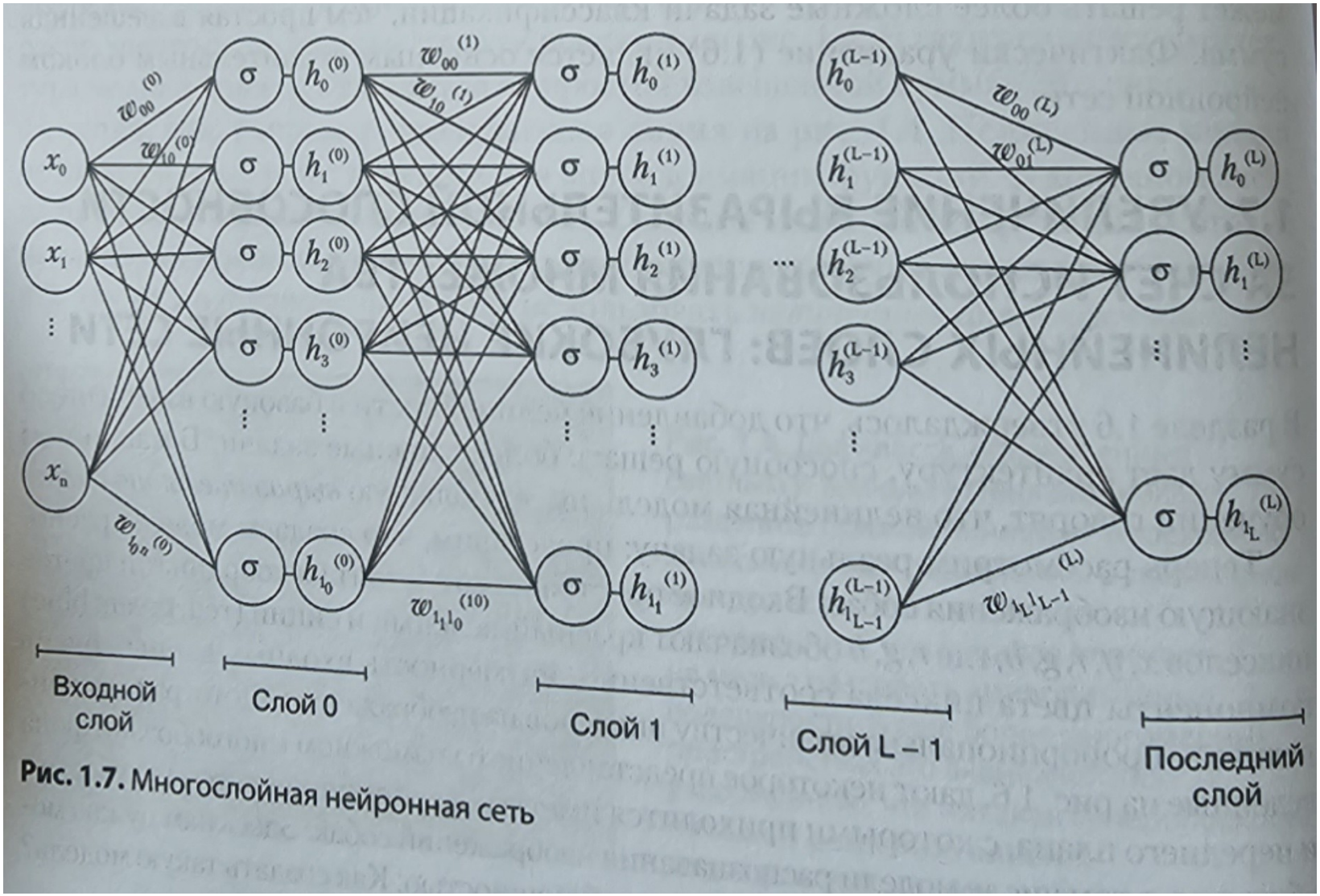
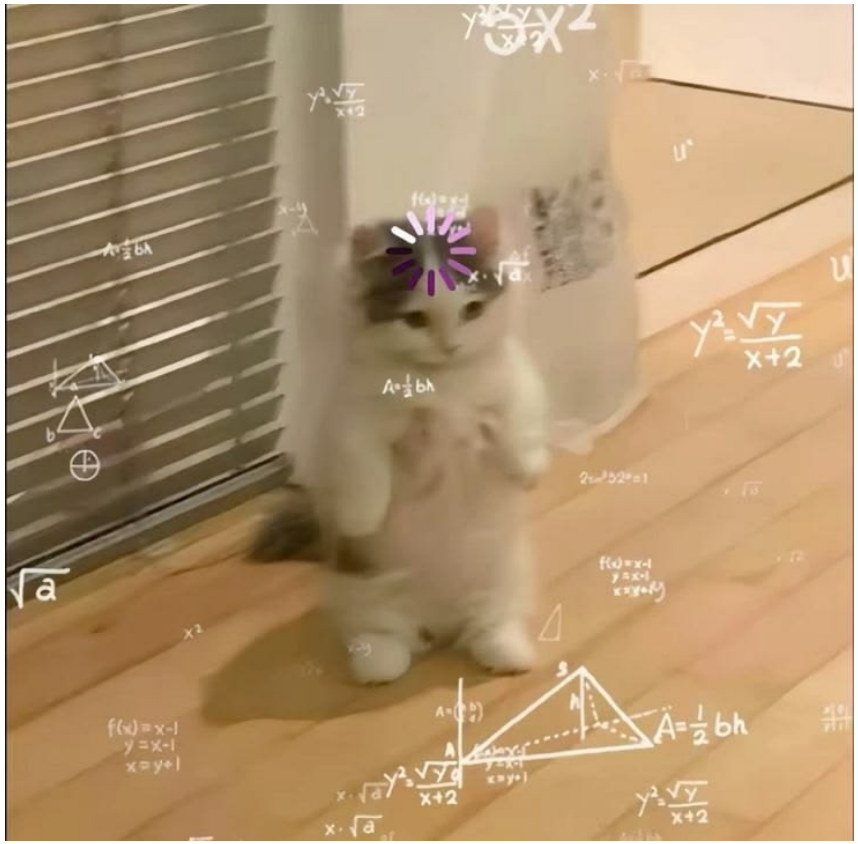


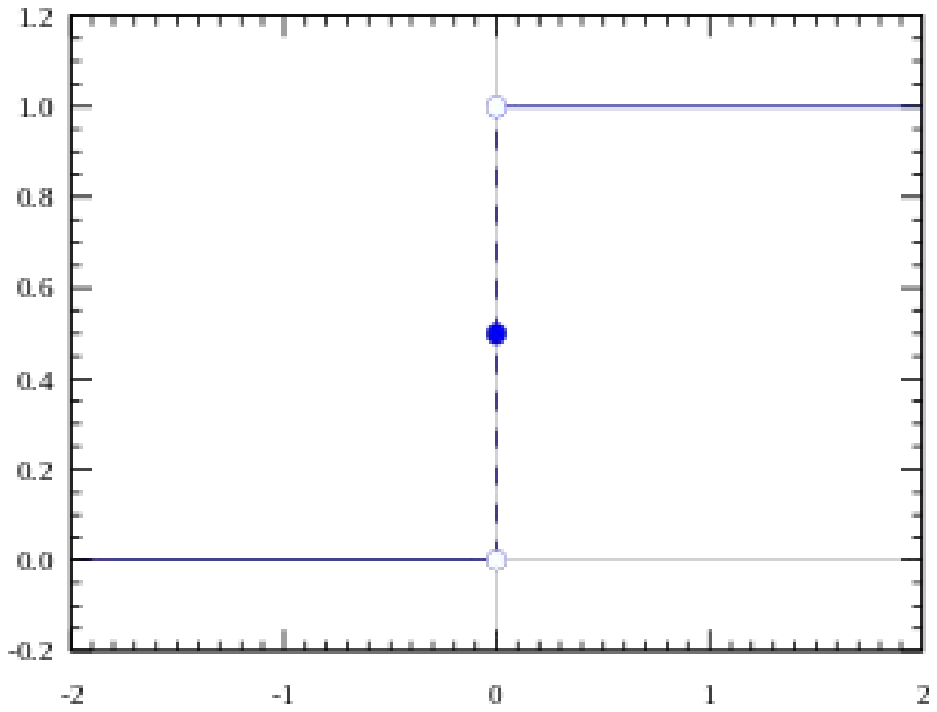
Рис. 1.7. Многослойная нейронная сеть

Функции активации



- Без функции активации нейрон — это просто линейная функция: output = w·x + b. А **композиция линейных функций — тоже линейная функция**. Это значит, что сколько бы нейронов мы ни соединяли в сеть, она всё равно останется линейной моделью и не сможет решать сложные задачи
- **Функция активации вносит нелинейность.**

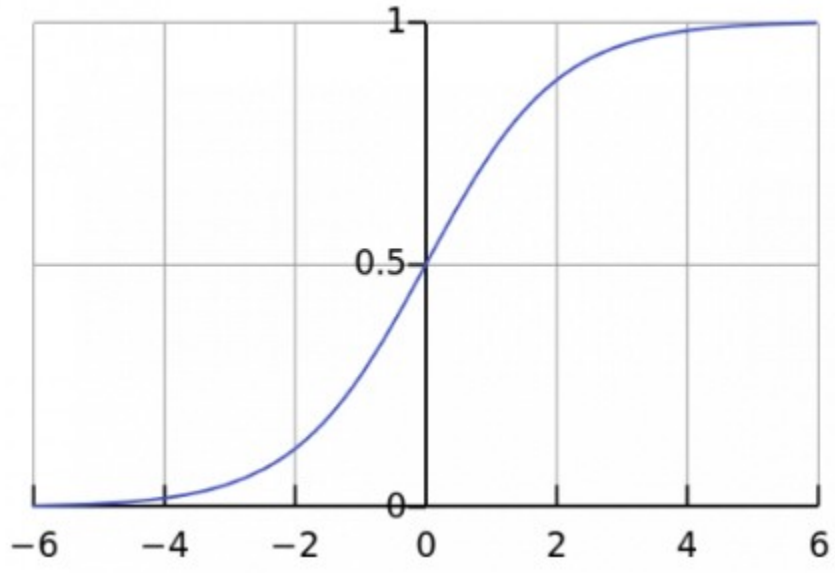
Ступенчатая функция



1) Функция А активирована, если Y > граница, иначе нет.
2) Не подходит для классификации многих классов

Sigmoid

- Сигмоида $A = \frac{1}{1+e^{-x}}$

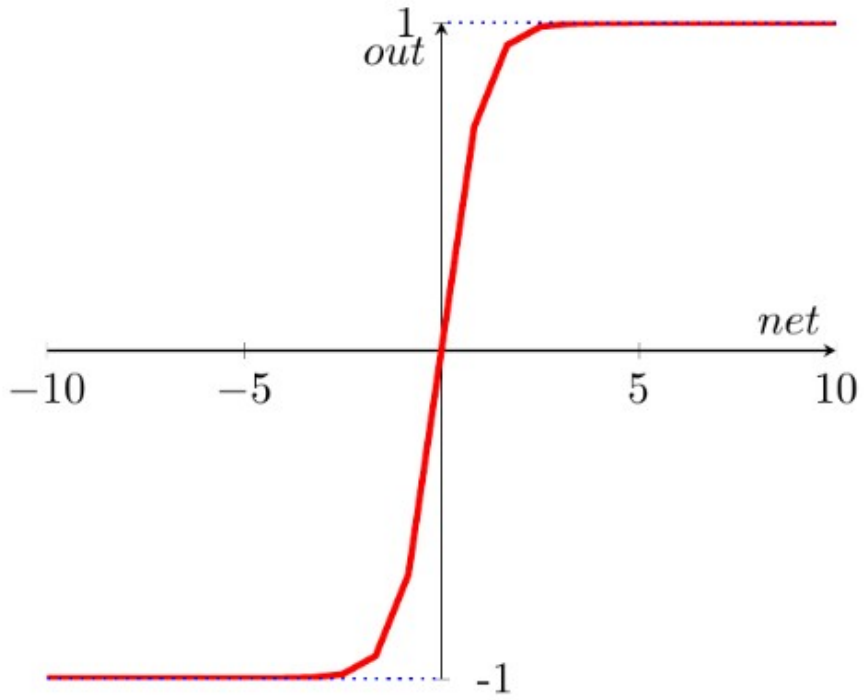


1) Диапазон (0,1)
2) Используется для выходного слоя бинарной классификации.
3) Интерпретируется как вероятность
4)Плавная и дифференцируемая
5) Минус: затухание градиента – при больших x производная близка к 0, сеть будет плохо обучаться

Tanh

- Гиперболический тангенс

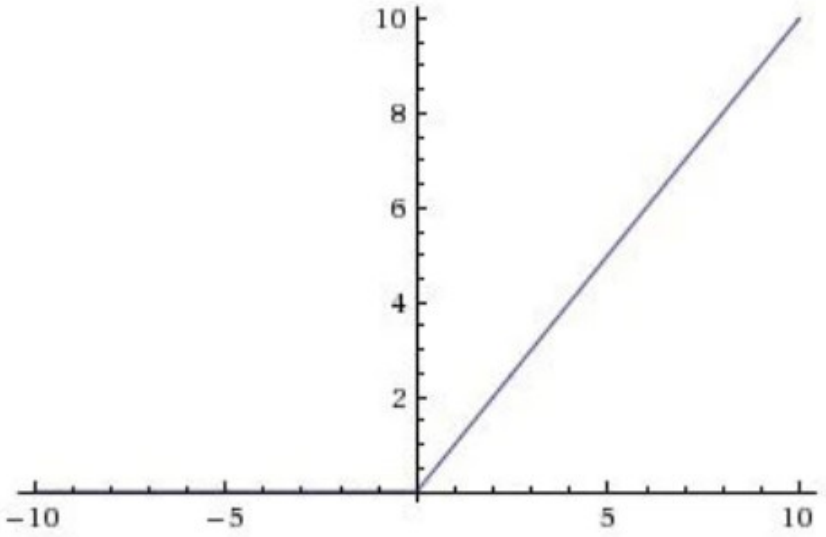
$$f(x) = tanh(x) = \frac{2}{1+e^{-2x}} - 1$$



1) Диапазон (-1, 1)
2) Используется, когда нужен центрированный выход
3) Градиенты сильнее, чем у сигмоиды
4) Но все еще страдаем от затухания градиента

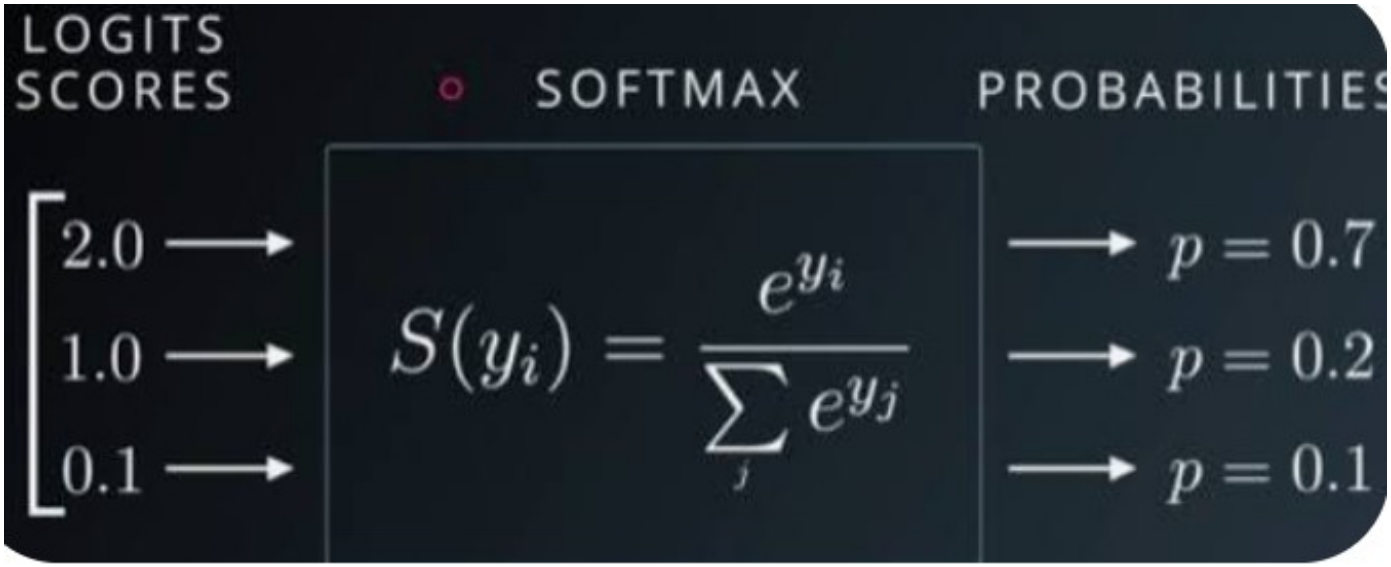
ReLu

- A(x) = max(0,x)



1) Диапазон [0, inf)
2) Используется в скрытых слоях
3) УРА! НЕТ затухания градиента
4) Быстрое вычисление
5) Минус: мертвые нейроны

Softmax



- 1) **Диапазон:** (0, 1), сумма всех выходов = 1.
- 2) Используется в выходном слое многоклассовой классификации (выдает вероятности для каждого класса)
- 3) Удобно интерпретировать
- 4) Минус: используется только для выходного слоя



КОДИМ !



Разбор кода в Google Collab

- Кусок кода в рамочке принято называть ячейкой. Во время работы нужно запускать все ячейки последовательно. Они независимы, но в нашем курсе почти все в рамках одного блока опираются на предыдущие

```python
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def tanh(x):
    return np.tanh(x)

def relu(x):
    return np.maximum(0, x)
```

Мы будем писать не весь код, а только значимые его части. Работу по визуализации и отображению (и прочую грязную работу мы оставим в стороне)

БЛОК 1. Нейрон

- Ячейка 1.
- NumPy — одна из основных библиотек Python для вычислений и работы с массивами и матрицами. Мы импортируем её, чтобы использовать в коде. np — это псевдоним (сокращённое имя), по которому мы будем к ней обращаться.
- matplotlib.pyplot — библиотека для построения графиков. Мы подключаем её под псевдонимом plt, чтобы потом вызывать нужные функции через короткое имя.
- def — ключевое слово для определения функции. После него указывается имя функции (например, sigmoid), затем в скобках — принимаемые параметры, после чего ставится двоеточие.
- Тело функции — то есть действия, которые выполняются при её вызове, — пишется с отступом в 4 пробела. В Colab отступы обычно ставятся автоматически.
- В этой ячейке определяются три функции активации: sigmoid, tanh и relu. Каждая из них принимает на вход число или массив x и возвращает результат по своей формуле. Позже мы сможем использовать их внутри нейронной сети.

```python
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def tanh(x):
    return np.tanh(x)

def relu(x):
    return np.maximum(0, x)
```

- Ячейка 2. Пишем класс нейрона

- class – это ключевое слово для объявления класса, который служит как бы шаблоном для всех конкретных нейронов, которые мы будем создавать на его основе.
- def __init__(...) – это специальный метод (конструктор), который автоматически вызывается в момент создания нового объекта-нейрона. В нём мы задаём, какие данные нужны, чтобы «оживить» нейрон:
  weights – список весов (по одному на каждый вход);
  bias – значение смещения;
  activation– имя функции активации.

- Внутри конструктора мы сохраняем переданные значения в атрибуты объекта (self.):
- self.weights = np.array(weights) – превращаем список весов в массив NumPy. Это нужно, чтобы потом удобно и быстро выполнять математические операции (например, умножение матриц).
- self.bias = bias – сохраняем смещение.
- self.activation = sigmoid if activation == 'sigmoid' else relu – выбираем функцию активации по имени. Если передано 'sigmoid', в атрибут попадёт функция sigmoid, иначе – relu. Важно: мы сохраняем саму функцию, а не результат её работы, чтобы вызывать её позже при обработке данных.

2. Класс нейрона

```
class Neuron:
    def __init__(self, weights, bias, activation):
        self.weights = np.array(weights)
        self.bias = bias
        self.activation = sigmoid if activation == 'sigmoid' else relu
```

- Дополняем функциональность нейрона
- def forward(self, inputs): – это метод, который описывает **прямой проход** (forward pass): то, как нейрон вычисляет свой выход по входным данным (т.е. считает взвешенную сумму). self – ссылка на конкретный объект-нейрон, а inputs – входные признаки (число или массив чисел).
- z = np.dot(inputs, self.weights) + self.bias – здесь считается взвешенная сумма:
- np.dot(inputs, self.weights) – скалярное произведение входа и весов. Если inputs – один вектор признаков, результат будет одним числом; если inputs – матрица (несколько примеров сразу), результатом будет массив значений для каждого примера.
- + self.bias – к полученной сумме прибавляется смещение. Оно сдвигает порог срабатывания нейрона, о чём мы говорили раньше.
- return self.activation(z) – к взвешенной сумме применяется функция активации, которая была сохранена в self.activation (например, sigmoid или relu). Именно это значение и возвращается как выход нейрона.

2. Класс нейрона

```python
class Neuron:
    def __init__(self, weights, bias, activation):
        self.weights = np.array(weights)
        self.bias = bias
        self.activation = sigmoid if activation == 'sigmoid' else relu
    def forward(self, inputs):
        z = np.dot(inputs, self.weights) + self.bias
        return self.activation(z)
```

3. Создание нейрона с одним входом

```
neuron1 = Neuron(weights=[0.5], bias=-0.2, activation='sigmoid')
```

- neuron1 = Neuron(weights=[0.5], bias=-0.2, activation='sigmoid') — здесь мы создаём конкретный нейрон, используя класс Neuron, который описали ранее.
- weights=[0.5] — список из одного веса. Это значит, что у нейрона один вход. Вес равен 0.5.
- bias=-0.2 — смещение отрицательное, то есть нейрон немного «притормаживает» срабатывание.
- activation='sigmoid' — в качестве функции активации выбрана сигмоида.
- Эти 3 параметра вы можете задать какие захочет ваша душа)

3. Создание нейрона с одним входом

```
neuron1 = Neuron(weights=[0.5], bias=-0.2, activation='sigmoid')
for x in [-2, -1, 0, 1, 2]:
    out = neuron1.forward([x])
    print(f"x = {x:2} -> выход = {out:.4f}")
```

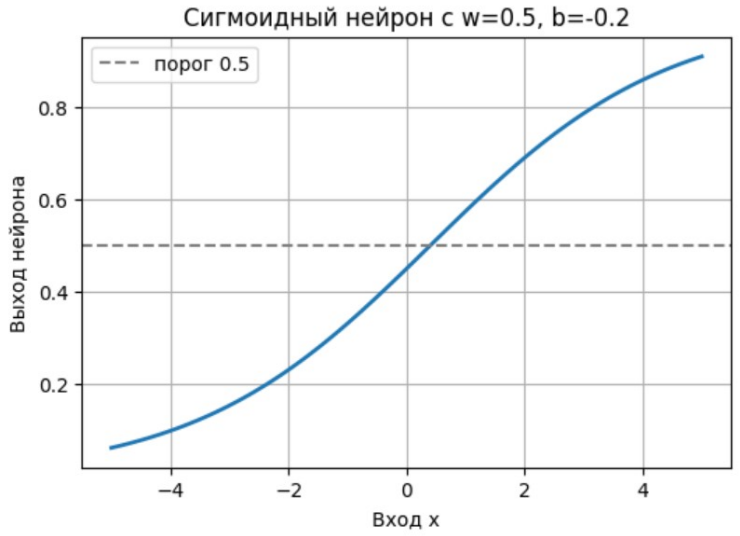
```
x = -2 -> выход = 0.2315
x = -1 -> выход = 0.3318
x =  0 -> выход = 0.4502
x =  1 -> выход = 0.5744
x =  2 -> выход = 0.6900
```

- for x in [-2, -1, 0, 1, 2]: — запускаем цикл по пяти тестовым значениям входа. Мы хотим посмотреть, как нейрон отреагирует на разные числа.
- out = neuron1.forward([x]) — вызываем у нейрона метод forward, передавая ему вход в виде списка [x]. Почему список? Потому что метод forward ожидает на вход массивоподобный объект (как раз для np.dot). Даже если признак один, мы оборачиваем его в список — так проще единообразно работать с массивами.
  Внутри forward происходит:
- np.dot([x], [0.5]) → x * 0.5
- Прибавляется смещение: x * 0.5 + (-0.2).
- Результат подаётся на сигмоиду: sigmoid(x * 0.5 - 0.2).
- rint(f"x = {x:2} -> выход = {out:.4f}") — печатаем строку в удобном формате.
- {x:2} — значение x, выровненное по правому краю шириной 2 символа.
- {out:.4f} — выходное значение с четырьмя знаками после запятой.
- Видно, что с ростом входа выход плавно увеличивается от 0.23 до 0.69. Это наглядная демонстрация того, как один нейрон с сигмоидой преобразует входное число: он не просто умножает на вес, а ещё и «сжимает» результат в диапазон от 0 до 1, делая его удобным для интерпретации как вероятность или степень уверенности.

- Далее идет ячейка с построением графика. Этот код мы не будем рассматривать подробно. Обсудим только результат его выполнения.
- Синяя кривая — это график функции активации нашего нейрона. По оси X откладывается входное значение (от -5 до 5), по оси Y — выход нейрона (от 0 до 1). Классическая S-образная кривая сигмоиды.
- Пунктирная линия на уровне 0.5 — это визуально обозначенный «порог принятия решения». Если выход нейрона выше 0.5, мы можем считать, что нейрон «уверен» в положительном ответе; если ниже — в отрицательном.
- Влияние веса (w) и смещения (b). На этом графике видно, как параметры w=0.5 и b=−0.2 формируют конкретный вид кривой:
  - **Смещение b=−0.2 сдвигает кривую вправо.** Это сдвигает момент, когда нейрон начнет склоняться к «положительному» решению. Если бы b было равно 0, точка пересечения с порогом 0.5 находилась бы ровно при x=0. Но из-за отрицательного смещения нейрону нужен чуть больший вход (примерно x=0.4), чтобы выдать 0.5. Т.е. мы задаем порог: когда нейрон решит, что пора склоняться к положительному ответу.
  - **Вес w=0.5 задает наклон.** Чувствительность нейрона к изменению входа, это резкость графика. Чем больше вес, тем «круче» была бы кривая (тем резче нейрон переключался бы между состояниями). Здесь наклон довольно пологий, поэтому переход от 0 к 1 занимает широкий диапазон значений X.

```
x_vals = np.linspace(-5, 5, 100)
y_vals = [neuron1.forward([x]) for x in x_vals]

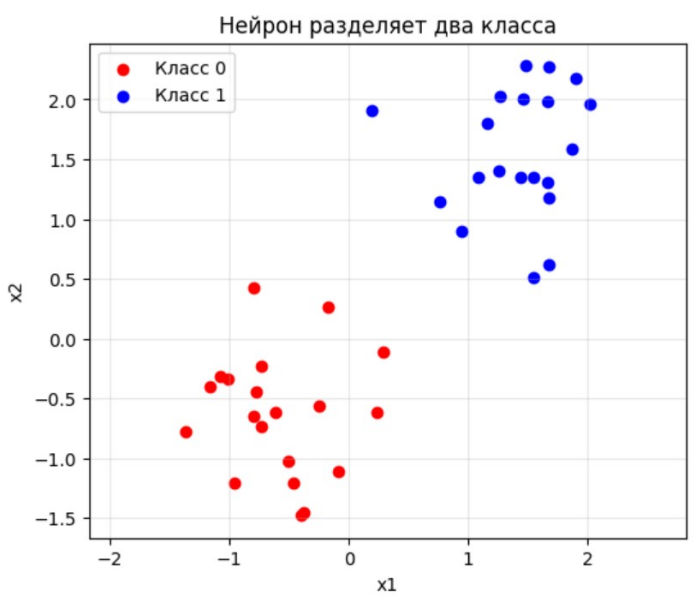
plt.figure(figsize=(6,4))
plt.plot(x_vals, y_vals, linewidth=2)
plt.axhline(0.5, color='gray', linestyle='--', label='порог 0.5')
plt.xlabel('Вход x')
plt.ylabel('Выход нейрона')
plt.title('Сигмоидный нейрон с w=0.5, b=-0.2')
plt.grid(True)
plt.legend()
plt.show()
```



- **Нейрон с двумя входами**
- Нейрон с двумя входами — это **линейный классификатор**. Он умеет делать только одно: проводить прямую линию, которая делит плоскость на две части.
- Данная ячейка генерирует линейно разделимые данные — то есть такие, которые можно разделить одной прямой. Мы создаём два облака точек — два класса.
- В следующей ячейке мы вручную зададим значения параметров нейрона (веса и смещение) так, чтобы его граница решения — прямая — разделила эти два класса.

[7]
0
сек.

```
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Нейрон разделяет два класса')
plt.legend()
plt.grid(alpha=0.3)
plt.axis('equal')  # чтобы масштаб по осям был одинаковым
plt.show()
```



- В следующей ячейке под комментарием (#Параметры нейрона) нужно создать 3 переменные: w1, w2, b – 2 веса и смещение соответственно.
- Ниже приведены значения, с которыми нейрон построит нужную прямую. Но вы можете поэкспериментировать и найти другую рабочую (или не очень) комбинацию.

- Здесь мы задаем всего 3 параметра, по сути описывающие прямую: w1·x1 + w2·x2 + b = 0
- Или

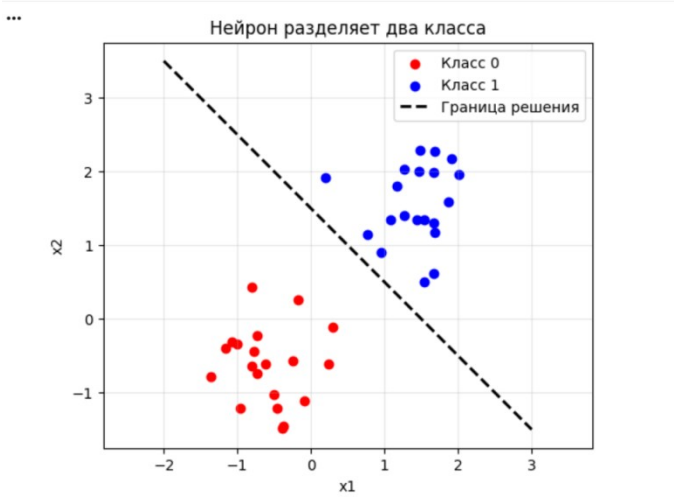
$$y = w1/w2 * x + b/w2$$

- Эта задача легко решается вручную – у нас всего 3 параметра. А если их 10 миллионов? Как они подбираются на самом деле – тема следующего урока.
- А сейчас мы своими усилиями получили работающий линейный классификатор, который разделяет 2 наших облака данных на 2 категории.
- Далее рассмотрим практическую ценность нашего нейрона

Задача: подобрать такие значения весов и смещения, чтобы получить корректную границу решения (разделяющую прямую)

```
# Параметры нейрона
w1, w2 = 1.0, 1.0
b = -1.5
```

- После задания параметров можно запускать ячейку. Результат будет подобным:



Рассмотрим практическую значимость проделанной работы. Вернёмся к примеру с милыми пушистиками.
Определим два признака:
x_1 — степень пушистости
x_2 — размер глаз
Что означают наши классы
Синее облако — милые животные (высокая пушистость + большие глаза)
Красное облако — менее милые (низкая пушистость + маленькие глаза)
Что мы получили
Нейрон нашёл границу решения — прямую, которая разделяет эти два класса. По одну сторону прямой — милые, по другую — не очень.
Как это использовать на практике
Теперь, когда появляется новое животное, мы:
Измеряем его пушистость (x_1) и размер глаз (x_2).
Отмечаем точку с этими координатами на плоскости.
Смотрим, по какую сторону от прямой она оказалась:

Попала в «синюю» область → животное милое.
Попала в «красную» область → не очень.

Итог

Наш нейрон превратился в работающий бинарный классификатор:

На входе — два числа (признака).

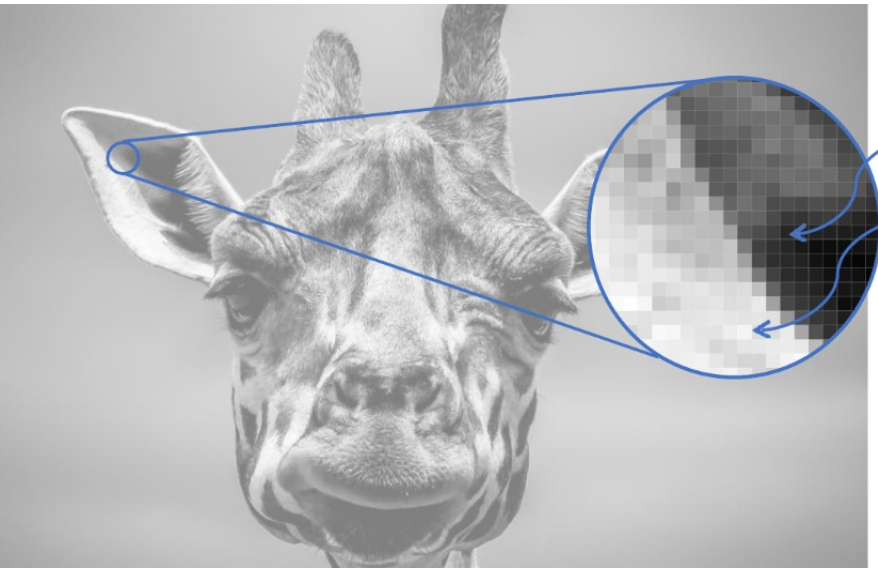
На выходе — решение: класс 0 или класс 1.

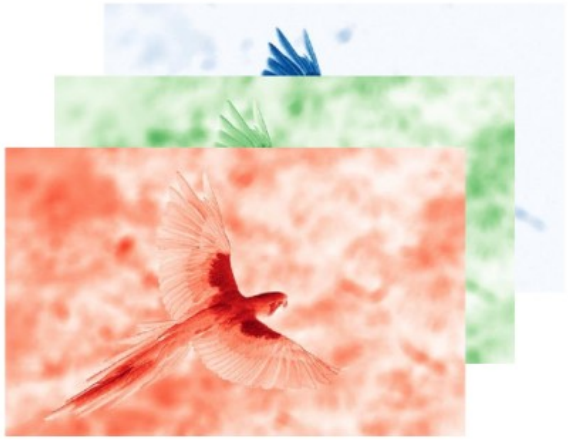
Именно так работают реальные ML-модели: принимают числовые признаки объекта и выдают решение. Разница лишь в том, что у реальных моделей признаков не 2, а тысячи, и граница решения — не прямая, а сложная поверхность.

- Мы увидели, что в компьютере все — числа
  Как впихать картинку?



Как компьютер работает с изображениями?





=

88	31	1	22
42	195	13	28
130	85	43	203
18	75	122	187
241	61	7	91
65	27	155	101



OpenCV
Снова кодить…

Первая ячейка — установка библиотеки OpenCV

0. Установка и импорт библиотек

In [ ]:
```
!pip install opencv-python matplotlib scikit-image
```

- Эта команда устанавливает три библиотеки: opencv-python, и scikit-image.
- Важно: установка происходит не на ваш компьютер, а в среду Google Colab — то есть в память того виртуального сервера, где выполняется блокнот. Ваше личное устройство при этом не задействуется, и после закрытия сессии Colab всё «забудет».
- Зачем это нужно: в Colab по умолчанию есть не все библиотеки. Команда !pip install добавляет их в текущую сессию, чтобы мы могли ими пользоваться.
- Что это за библиотеки:
- opencv-python — самая популярная библиотека для работы с изображениями и видео. Основа компьютерного зрения.
- scikit-image — библиотека с набором готовых изображений и инструментов для их обработки.

Вторая ячейка — импорт библиотек

- Здесь мы подключаем нужные библиотеки к нашему блокноту, чтобы обращаться к ним по коротким именам.
- numpy as np — библиотека для работы с массивами чисел (наши матрицы изображений будут именно такими).
- matplotlib.pyplot as plt — для отображения изображений и графиков.
- cv2 — это и есть установленная OpenCV.
- from skimage import data — импортируем готовые встроенные изображения (например, data.astronaut(), data.camera()), чтобы нам не пришлось загружать свои файлы.

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from skimage import data
```

Загрузка изображения

1. Загрузка изображения Загружаем встроенное изображение и выводим на экран

Варианты: astronaut, cat, coffee, retina, ограничимся)

```
img_rgb = data.cat()  # Это RGB-изображение 512x512x3
# Показываем изображение
plt.figure(figsize=(6,6))  # 1. Создаем квадратный холст 6x6 дюймов
plt.imshow(img_rgb)        # 2. Рисуем на нем наше изображение
plt.title("Оригинал")      # 3. Добавляем заголовок
plt.axis('off')            # 4. Убираем оси координат
plt.show()                 # 5. Показываем результат
```

Оригинал



```
img_rgb = data.coffee()  # Это RGB-изображение 512x512x3
# Показываем изображение
plt.figure(figsize=(6,6))  # 1. Создаем квадратный холст 6x6 дюймов
plt.imshow(img_rgb)        # 2. Рисуем на нем наше изображение
plt.title("Оригинал")      # 3. Добавляем заголовок
plt.axis('off')            # 4. Убираем оси координат
plt.show()                 # 5. Показываем результат
```

Оригинал



- Разберем первую строчку кода ячейки
- img_rgb = data.coffee() – вместо кофе вы можете вписать свой вариант (astronaut, cat, coffee, retina, ограничимся)) и выбрать себе полюбившееся изображения для дальнейшей работы в этом блокноте

- `img_rgb = data.coffee() # загружаем изображение`

- Здесь мы загружаем изображение из библиотеки skimage.
  Команда data.coffee() возвращает картинку с чашкой кофе — она уже встроена в библиотеку, ничего скачивать не нужно.
- **Что можно выбрать вместо кофе:**
- В skimage.data есть несколько готовых изображений. Вот самые популярные:
- data.coffee() - Чашка кофе
- data.astronaut() - Фотография астронавта
- data.cat() или data.chelsea() — Кошка
- data.retina() - Снимок сетчатки глаза

- Следующие 5 строчек ячейки — отображение загруженного изображения средствами plt

- В следующей ячейке дописывать ничего не нужно, но мы разберем ее код и посмотрим, что он делает

[4]
0
сек.

```
print(f"Размер изображения: {img_rgb.shape}")  # (512, 512, 3)
print(f"Тип данных: {img_rgb.dtype}")          # uint8
print(f"Минимальное значение пикселя: {img_rgb.min()}, максимальное: {img_rgb.max()}")
```

```
Размер изображения: (400, 600, 3)
Тип данных: uint8
Минимальное значение пикселя: 0, максимальное: 255
```

Img_rgb – название переменной, в которой мы храним загруженное изображение. Под этой переменной скрываются три матрицы чисел – по одной на каждый цветовой канал (R, G, B). Вместе они образуют трёхмерный тензор.
Почему трёхмерный?
- Первое измерение – высота изображения (сколько строк пикселей).
- Второе измерение – ширина (сколько пикселей в строке).
- Третье измерение – каналы цвета: 0-й – красный, 1-й – зелёный, 2-й – синий.

То есть img_rgb[i, j, k] – это значение k-го канала в пикселе, который находится в i-й строке и j-м столбце

.shape / .dtype / .min()/ .max() – атрибуты массива np

Атрибут – это характеристика объекта, к которой обращаются через точку. У массива NumPy есть набор встроенных атрибутов, которые описывают сам массив: его форму, тип данных, размер и т.д.

- Этот код выводит три ключевых параметра изображения, которые помогают понять его структуру:
- **img_rgb.shape — размерность массива.** Результат (400, 600, 3) означает: 400 пикселей в высоту, 600 в ширину и 3 цветовых канала (R, G, B). Примечание: в комментарии указано (512, 512, 3) — это пример для astronaut(), а для coffee() (или вашего выбранного изображения) размеры будут другими, например, (400, 600, 3).
- **img_rgb.dtype** — тип данных элементов массива. uint8 означает «беззнаковое целое 8 бит» — то есть числа от 0 до 255. Это стандартный формат для изображений.
- **img_rgb.min() и img_rgb.max()** — минимальное и максимальное значение пикселя во всём изображении. Здесь 0 — это самый тёмный пиксель (чёрный), а 255 — самый яркий (белый). Эти значения показывают, насколько полно изображение использует доступный диапазон яркости.

2.Изображение как матрица чисел

```
print("Первые 2x1 пикселей (верхний левый угол):")
print(img_rgb[:2, :1])   # форма (2,1,3) — каждый пиксель — три числа (R,G,B)
```

```
Первые 2x1 пикселей (верхний левый угол):
[[[21 13  8]]

 [[21 13  7]]]
```

- Следующая ячейка позволяет посмотреть на изображение глазами компьютера — выведем 2 пикселя, каждый из них — массив из трех чисел (R, G,B).

- Теперь непосредственно к коду:
- Ключевой момент — **срез (slice)** [:2, :1].
- В NumPy срезы работают по формуле [start:stop], где start включается, а stop — нет. Если не указан start, Python поставит 0, т.е. начать с самого начала. Если не указан stop, Python поставит послений индекс, т.е. идти до самого конца.

- :2 — это сокращение от 0:2. Значит: «взять строки с 0-й по 1-ю включительно» (то есть 2 строки).
- :1 — это сокращение от 0:1. Значит: «взять столбец с 0-й по 0-ю» (то есть 1 столбец).
- Третья ось (цветовые каналы) не указана, поэтому берутся все три канала (R, G, B).
- Итог: мы получаем кусочек изображения размером 2 строки × 1 столбец × 3 канала. Это форма (2, 1, 3).

2.Изображение как матрица чисел

```
print("Первые 2x1 пикселей (верхний левый угол):")
print(img_rgb[:2, :1])   # форма (2,1,3) — каждый пиксель — три числа (R,G,B)
```

```
Первые 2x1 пикселей (верхний левый угол):
[[[21 13  8]]

 [[21 13  7]]]
```

- Как читать вывод?
- Разберём по уровням вложенности:

  Внешние скобки — это весь наш срез.

  Средние скобки [[...]] — это одна строка пикселей (у нас их две).

  Внутренние скобки [21 13 8] — это один пиксель, состоящий из трёх чисел: R=21, G=13, B=8.
- Что это значит на практике:

  Цвет (21, 13, 8) — это очень тёмный коричневый (почти чёрный).

  R немного больше G и B — значит, в этом пикселе преобладает красноватый оттенок.

- Код следующей ячейки — функция, которую мы далее будем использовать для отображения нескольких изображений в ряд, эту ячейку тоже надо запустить, чтобы среда знала о ее существовании.
- **def** — ключевое слово, объявляющее функцию.
- **show_images** — имя функции.
- **images** — первый параметр: список изображений (массивов NumPy), которые нужно отобразить.
- **titles** — второй параметр: список строк с заголовками. Должен быть той же длины, что и images.
- **cols=3** — третий параметр: количество колонок в сетке графиков. Значение по умолчанию — 3 (если не указать иное).
- **Что делает функция:** создаёт фигуру с несколькими подграфиками в ряд, отображает каждое изображение и подписывает его заголовком. Если изображение чёрно-белое (двумерное), оно выводится с cmap='gray', если цветное — как есть.

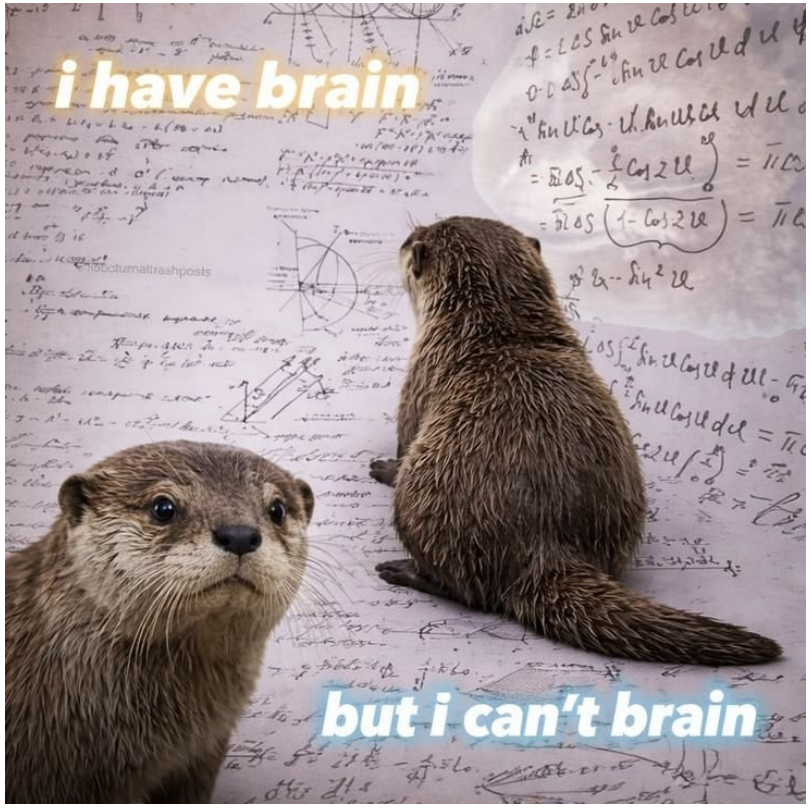
```
# Создаём функцию для отображения нескольких изображений в ряд
def show_images(images, titles, cols=3):
    fig, axes = plt.subplots(1, cols, figsize=(15, 5))
    for i, (img, title) in enumerate(zip(images, titles)):
        if len(img.shape) == 2:
            axes[i].imshow(img, cmap='gray')
        else:
            axes[i].imshow(img)
        axes[i].set_title(title)
        axes[i].axis('off')
    plt.tight_layout()
    plt.show()
```

Пример вызова:

- img_rgb, flip_h, flip_v –переменные, в которых хранятся массивы изображений.
- [img_rgb, flip_h, flip_v] – список этих изображений, который передается в функцию. порядок в списке соответствует порядку подписей в titles.
- ["Оригинал", "Отражение по горизонтали", "Отражение по вертикали"] – список подписей для картинок при отображении
- cols=3 – число столбцов в сетке

```
show_images([img_rgb, flip_h, flip_v],
            ["Оригинал", "Отражение по горизонтали", "Отражение по вертикали"], cols=3)
```

- Далее мы будем рассматривать геометрические преобразования, масштабирование и обрезку.
- Зачем нам это нужно? Ответ — Аугментация



Аугментация

- Аугментация — это искусственное расширение датасета путём создания изменённых версий существующих изображений. Мы не собираем новые фото, а модифицируем уже имеющиеся, сохраняя их ключевые признаки.
- Зачем это нужно:

  Защита от переобучения → модель не запоминает конкретные картинки, а учится на вариациях.

  Больше данных → модель лучше учится.

  Устойчивость к изменениям → модель учится распознавать объект под разными углами, при разном освещении, в разном масштабе.

- Примеры аугментации: отражение, поворот, масштабирование, сдвиг, изменения яркости, добавление шума (небольшие искажения пикселей)

- Важно: при аугментации ключевые признаки должны сохраняться. Например:
- Фото кошки можно отразить, повернуть, изменить яркость — она останется кошкой. А если перевернуть изображение цифры «6», получится «9». Для задачи распознавания цифр это уже не аугментация, а искажение метки — так делать нельзя.

Отражение по горизонтали и вертикали

- Следующая ячейка демонстрирует операцию отражения.
- Функция cv2.flip принимает 2 параметра:
  - Изображение – массив NumPy, который нужно отразить.
  - flipCode – код отражения:
    - 1 – отражение по горизонтали (лево ↔ право),
    - 0 – отражение по вертикали (верх ↔ низ),
    - -1 – отражение по обеим осям одновременно (поворот на 180°).

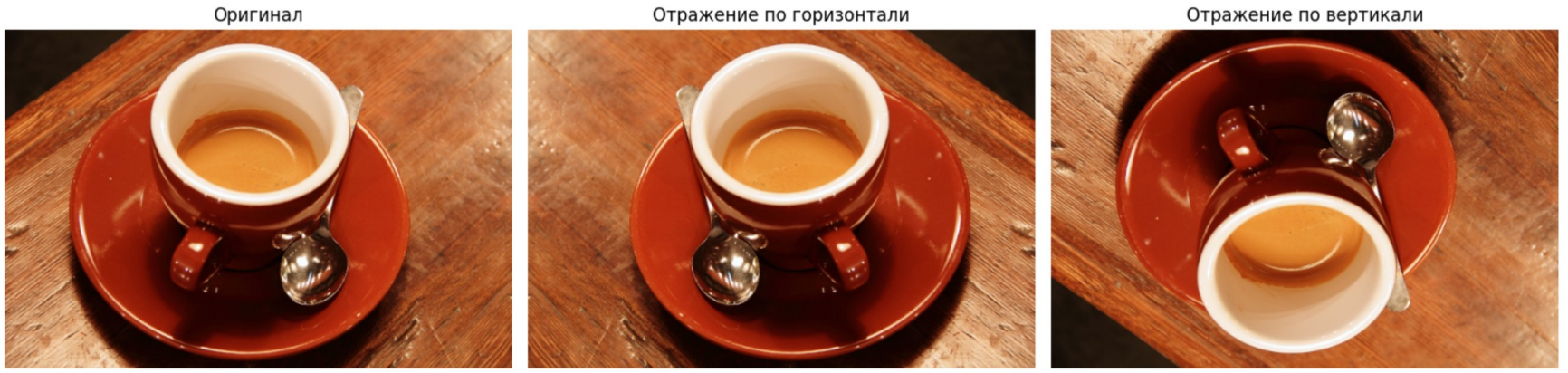
3. Базовые геометрические преобразования

```
# 3.1. Отражение по горизонтали и вертикали
flip_h = cv2.flip(img_rgb, 1)   # 1 – по горизонтали
flip_v = cv2.flip(img_rgb, 0)   # 0 – по вертикали
```

3. Базовые геометрические преобразования

```python
# 3.1. Отражение по горизонтали и вертикали
flip_h = cv2.flip(img_rgb, 1)   # 1 — по горизонтали
flip_v = cv2.flip(img_rgb, 0)   # 0 — по вертикали

show_images([img_rgb, flip_h, flip_v],
            ["Оригинал", "Отражение по горизонтали", "Отражение по вертикали"], cols=3)
```



Поворот на 90°, 180°, 270°

```
# 3.2. Поворот на 90°, 180°, 270°
rot_90 = cv2.rotate(img_rgb, cv2.ROTATE_90_CLOCKWISE)
rot_180 = cv2.rotate(img_rgb, cv2.ROTATE_180)
rot_270 = cv2.rotate(img_rgb, cv2.ROTATE_90_COUNTERCLOCKWISE)

show_images([rot_90, rot_180, rot_270],
            ["Поворот 90°", "Поворот 180°", "Поворот 270°"], cols=3)
```

- Функция **cv2.rotate** принимает два аргумента:
  1 **Изображение** — массив NumPy, который нужно повернуть.
  2 **Код поворота** — одна из встроенных констант OpenCV:

cv2.ROTATE_90_CLOCKWISE - Поворот на 90° по часовой стрелке (Вправо)
cv2.ROTATE_180 - Поворот на 180° (Переворот)
cv2.ROTATE_90_COUNTERCLOCKWISE - Поворот на 90° против часовой стрелки (Влево)

- Важный нюанс: при повороте на 90° и 270° ширина и высота меняются местами. Если исходное изображение было 400×600, после поворота оно станет 600×400. При повороте на 180° размеры остаются прежними.
- Порядок каналов не важен: cv2.rotate работает с массивом как с набором пикселей и не зависит от того, RGB это или BGR. Она просто переставляет строки и столбцы.

```python
# 3.2. Поворот на 90°, 180°, 270°
rot_90 = cv2.rotate(img_rgb, cv2.ROTATE_90_CLOCKWISE)
rot_180 = cv2.rotate(img_rgb, cv2.ROTATE_180)
rot_270 = cv2.rotate(img_rgb, cv2.ROTATE_90_COUNTERCLOCKWISE)

show_images([rot_90, rot_180, rot_270],
            ["Поворот 90°", "Поворот 180°", "Поворот 270°"], cols=3)
```

Поворот 90°



Поворот 180°



Поворот 270°



Масштабирование: сжатие и увеличение

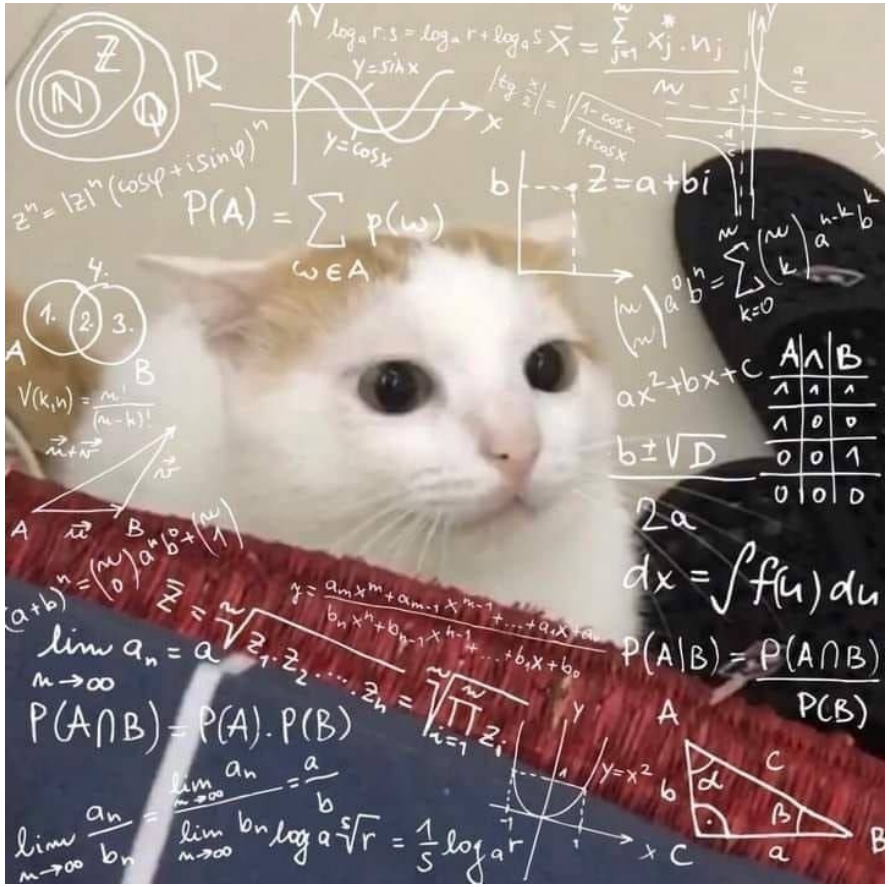
- Функция cv2.resize принимает три аргумента (третий – необязательный):

  1) Изображение – массив NumPy, который нужно масштабировать.

  2) dsize – новый размер в формате (ширина, высота). Важно: сначала ширина, потом высота – именно в таком порядке, в отличие от .shape, где сначала высота!

  3) interpolation – метод интерполяции (как заполнять новые пиксели). По умолчанию – cv2.INTER_LINEAR.

Масштабирование: сжатие и увеличение



- В cv2.resize() третий **аргумент (interpolation)** определяет, как именно будут вычисляться новые пиксели. По сути, это правила, по которым программа «додумывает» цвета на новом изображении, основываясь на старом. Давайте разберем каждый метод.

Методы

- 1) INTER_LINEAR – билинейная интерполяция (по умолчанию)
- Суть: новый пиксель – взвешенное среднее четырех ближайших пикселей

- 2)INTER_AREA – пересчет по площади (для уменьшения)

Суть: при уменьшении новый пиксель покрывает собой область из старых пикселей, алгоритм вычисляет среднее взвешенное по этой области.

- 3) INTER_CUBIC – бикубическая интерполяция (для увеличения)

Суть: для вычисления нового пикселя используется 16 ближайших соседей, а не 4. Учитывается не только расстояние, но и тренд изменения цвета.

4. Изменение размера (масштабирование)

```python
# Увеличение изображения в 2 раза
height, width = img_rgb.shape[:2]
resized_large = cv2.resize(img_rgb, (width*2, height*2), interpolation=cv2.INTER_CUBIC)
```

- height, width = img_rgb.shape[:2]
- img_rgb.shape возвращает кортеж (высота, ширина, каналы) — например, (400, 600, 3).
- [:2] берёт только первые два элемента: высоту и ширину.
- Результат распаковывается в две переменные: height = 400, width = 600. Эти переменные мы будем использовать, чтобы посчитать новый размер

- Функция cv2.resize принимает три аргумента:
  - img_rgb — масив Numpy, с нашим изображением, которое будем масштабировать
  - dsize - новый размер в формате (ширина, высота). У нас (width*2, height*2) = (1200, 800)
  - Interpolation - метод интерполяции (как заполнять новые пиксели)

[13]
✓ 1 сек.

```python
# Увеличение изображения в 2 раза
height, width = img_rgb.shape[:2]
resized_large = cv2.resize(img_rgb, (width*2, height*2), interpolation=cv2.INTER_CUBIC)

# Показываем только увеличенное изображение
plt.figure(figsize=(10, 10))
plt.imshow(resized_large)
plt.title(f"Увеличенное в 2 раза (размер: {resized_large.shape[1]}x{resized_large.shape[0]})")
plt.axis('off')
plt.show()
```



Обрезка

- Хотим вырезать центральную часть изображения размером **200×200 пикселей**. Это равносильно взятию среза из изображения по осям высоты и ширины — каналы не трогаем (берутся все три).
- Чтобы понять, какие границы у нашего среза, определим координаты. Нам нужно найти середину изображения:
- width//2 — середина по горизонтали (оператор // — целочисленное деление).
- height//2 — середина по вертикали.
- Затем отступаем по 100 пикселей в каждую сторону:

```
# Вырежем центральную часть размером 200x200 пикселей
x1 = width//2 - 100
x2 = width//2 + 100
y1 = height//2 - 100
y2 = height//2 + 100
cropped = img_rgb[y1:y2, x1:x2]
```

```python
# Вырежем центральную часть размером 200x200 пикселей
x1 = width//2 - 100
x2 = width//2 + 100
y1 = height//2 - 100
y2 = height//2 + 100
cropped = img_rgb[y1:y2, x1:x2]

plt.figure(figsize=(4,4))
plt.imshow(cropped)
plt.title("Обрезка центра 200x200")
plt.axis('off')
plt.show()
```

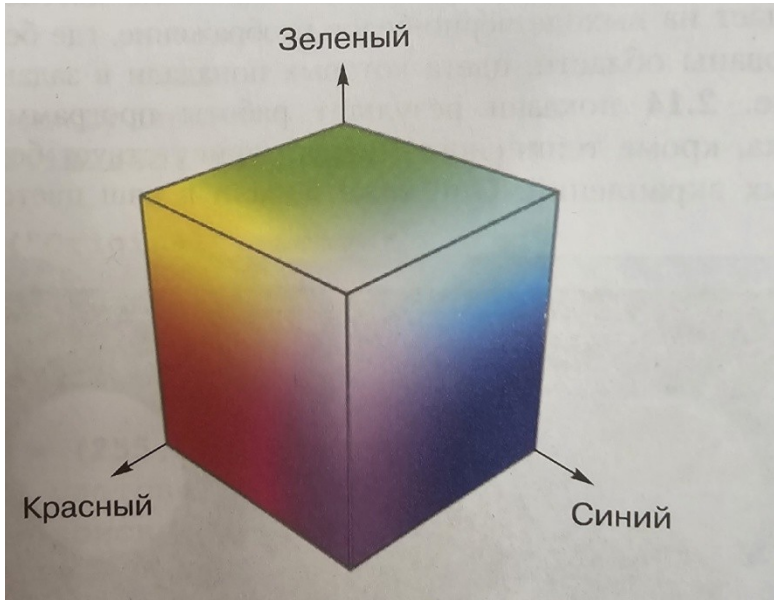


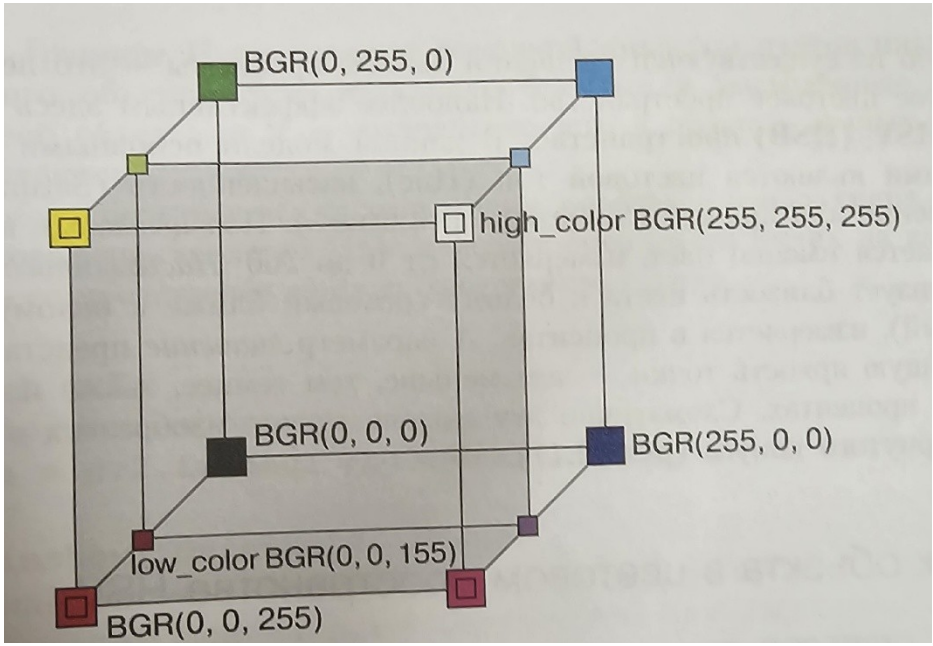
Цветовые пространства

- **Цветовое пространство** — это способ описать цвет с помощью набора чисел. Один и тот же цвет можно представить по-разному, в зависимости от того, какую модель мы используем.
- Самые известные:
- **RGB** — красный, зелёный, синий (так работают мониторы).
- **HSV** — тон, насыщенность, яркость (удобно для выделения цвета).

- **RGB – куб.**

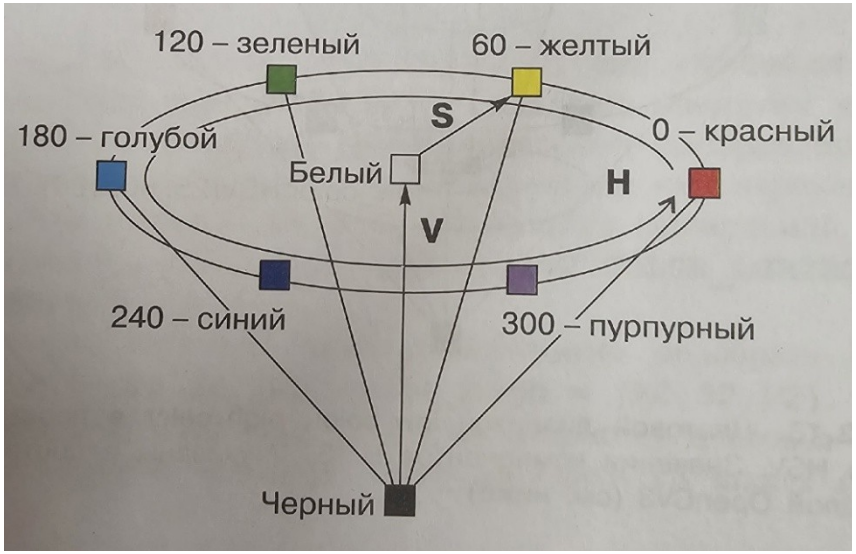
- Представьте три оси: по одной откладывается красный, по другой – зелёный, по третьей – синий. Каждая ось – это шкала от 0 до 255.
- Любой цвет – это точка внутри куба, которую задают три координаты: (R, G, B).
- Углы куба – крайние случаи: (0, 0, 0) – чёрный, (255, 255, 255) – белый, (255, 0, 0) – чистый красный и так далее.

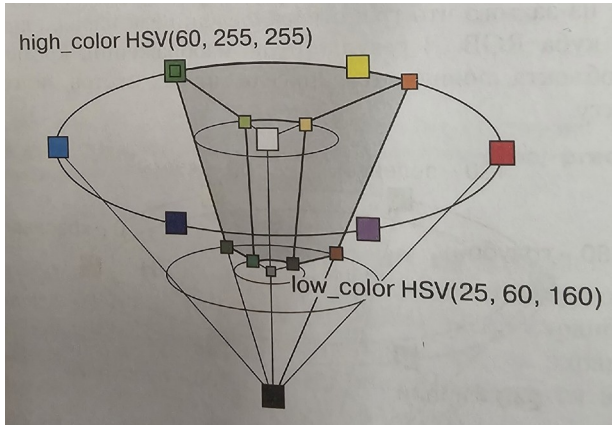




Пространство HSV

- Hue – тон (цвет от 0 до 180)
- Saturation – насыщенность (близость к белому, %)
- Value – яркость (чем меньше, тем темнее, %)





На картинке слева показана форма HSV – это **конус**:
Основание (круг) – все возможные тона (Hue) при максимальной насыщенности.
Ось снизу вверх – Value (от чёрного к яркому).
От центра к краю – Saturation (от серого к чистому цвету).
Крайние случаи:
Чёрный – V = 0, любой H и S.
Белый – S = 0, V = 100%.
Серый – S = 0, любое V.

- **Hue — тон (0–180 в OpenCV)**
- Это угол на цветовом круге: 0° — красный, 60° — жёлтый, 120° — зелёный, 180° — голубой, 240° — синий, 300° — пурпурный.
- **Важно:** в OpenCV Hue хранится в диапазоне **0–180**, а не 0–360. Причина — тип данных uint8 (от 0 до 255). Чтобы угол 360° уместился в эти границы, его делят на 2. То есть реальный угол = H × 2.

- **Saturation — насыщенность (0–255, или 0–100%)**
- Показывает, насколько цвет «чистый» или разбавленный белым:
- **S = 100%** — чистый, насыщенный цвет.
- **S = 50%** — полутон (например, розовый вместо красного).
- **S = 0%** — серый (цвета нет).

- **Value — яркость (0–255, или 0–100%)**
- Показывает, насколько цвет светлый или тёмный:
- **V = 100%** — максимально яркий.
- **V = 50%** — приглушённый (тёмно-красный вместо красного).
- **V = 0%** — чёрный.
- **Ключевое различие:** S «разбавляет» цвет **белым**, а V — **чёрным**. Это два независимых параметра.

Зачем менять цветовое пространство

- Основная причина — избавиться от шумов.
- Разные пространства разделяют информацию по-разному:
- В RGB цвет и яркость «переплетены»: чтобы понять, какой это цвет, нужно смотреть все три канала.
- В HSV цвет (Hue) отделён от яркости (Value). Это значит, что мы можем, например, выделить объект по цвету, не завися от освещения.

6. Работа с цветовыми пространствами

Важно: OpenCV использует порядок BGR, а skimage — RGB. Если хотите показать изображение через matplotlib (plt.imshow) — нужно BGR → RGB, потому что matplotlib ожидает RGB.

```
# Преобразуем в BGR (как в OpenCV)
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
```

- В следующей ячейке вам предлагается перевести изображение из одного цветового пространства в другое. Это выполняется с помощью функции **cv2.cvtColor**.
- В качестве аргументов передаётся:
- **Исходное изображение** — массив NumPy, который нужно преобразовать.
- **Код преобразования** — константа OpenCV, указывающая, из какого пространства в какое мы переходим.

Самые популярные коды:

- cv2.COLOR_RGB2BGR - Меняет местами красный и синий каналы (RGB → BGR)
- cv2.COLOR_BGR2RGB - Обратное преобразование (BGR → RGB)
- cv2.COLOR_RGB2GRAY - Переводит в оттенки серого (RGB → Grayscale)
- cv2.COLOR_BGR2GRAY - То же, но из BGR
- cv2.COLOR_BGR2HSV - Переводит в HSV (BGR → HSV)
- cv2.COLOR_RGB2HSV - То же, но из RGB

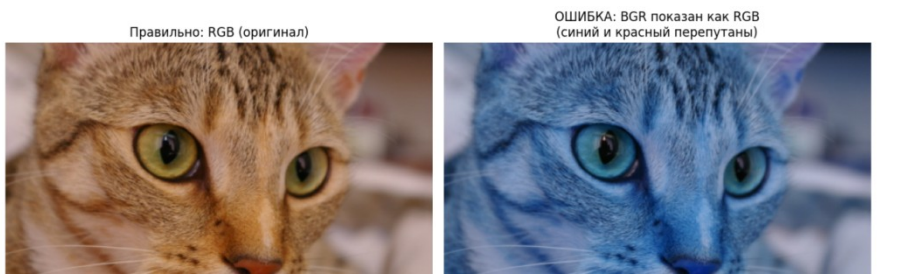
Важно: код преобразования всегда указывает **направление**: «из чего» → «во что». Если перепутать, результат будет неправильным.

- Эта ячейка также демонстрирует, что произойдёт, если не преобразовать изображение перед выводом через matplotlib.
- Мы загрузили картинку через skimage — она в **RGB**. Если применить cv2.cvtColor с кодом COLOR_RGB2BGR, каналы переставятся, и массив станет **BGR**.
- А plt.imshow **всегда ожидает RGB**. Она не знает, что мы подсунули ей BGR, и честно интерпретирует первое число как «красный», второе — как «зелёный», третье — как «синий».
- **В результате:**
- Реальный красный канал (последний в BGR) читается как синий → рыжая кошка выглядит синей.
- Реальный синий канал (первый в BGR) читается как красный.
- **Чтобы избежать ошибки**, перед выводом через plt.imshow нужно конвертировать изображение обратно:
- img_rgb_back = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
- plt.imshow(img_rgb_back)

```
# Преобразуем в BGR (как в OpenCV)
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Правильно: RGB (оригинал)")
plt.axis('off')

# ОШИБКА: показываем BGR как RGB (цвета искажены)
plt.subplot(1, 2, 2)
plt.imshow(img_bgr)    # неверно: BGR интерпретируется как RGB
plt.title("ОШИБКА: BGR показан как RGB\n(синий и красный перепутаны)")
plt.axis('off')
plt.tight_layout()
plt.show()
```

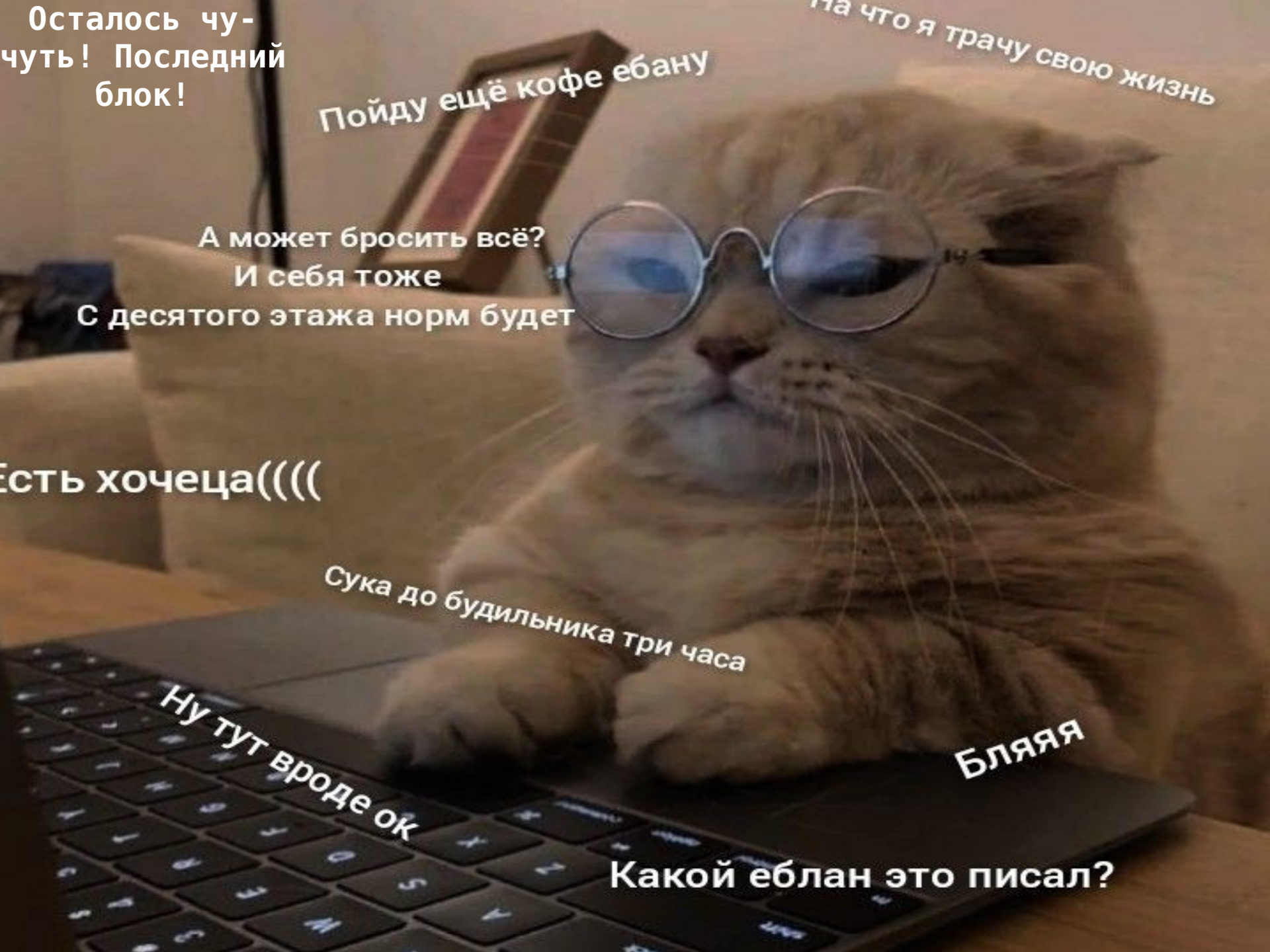


- Следующая ячейка демонстрирует пример перевода загруженного в **RGB изображения в оттенки серого и отображение этой картинки средствами библиотеки** ma

```
# Правильный перевод в Grayscale из RGB (без лишней конвертации)
img_gray_correct = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)

plt.figure(figsize=(15, 5))
plt.imshow(img_gray_correct, cmap='gray')
plt.title("Правильно: RGB -> Grayscale")
plt.axis('off')
plt.show()
```




Осталось чу-
чуть! Последний
блок!
Пойду ещё кофе ебану
На что я трачу свою жизнь
А может бросить всё?
И себя тоже
С десятого этажа норм будет
Есть хочеца((((
Сука до будильника три часа
Ну тут вроде ок
Бляяя
Какой еблан это писал?

Цветовые маски

- **Цветовая маска** – это инструмент для одной из основных задач компьютерного зрения: **детекции объекта, т.е. его обнаружения на изображении. В данном случае по цвету**.
- **Как это работает:**
- Задаём **диапазон цвета** искомого объекта (обычно в пространстве HSV, потому что там цвет отделён от яркости).
- Программа просматривает **каждый пиксель** изображения.
- Если все каналы пикселя попадают в заданный диапазон – пиксель считается «подходящим» (предположительно принадлежит объекту).
- В изображении, которое мы получаем после применения маски такой пиксель становится **белым**, остальные – **чёрными**.
- **Результат:** чёрно-белое изображение, где белые области – это объекты нужного цвета. По такой маске объект найти гораздо проще: можно наложить её на оригинал, найти контуры или вычислить ограничивающий прямоугольник.

Пример поиска объекта красного цвета



- Реализуем!

7. Цветовые маски

Поиск объекта по цвету. Цветовая маска при наложении на исходное изображение отфильтровывает цвета по диаапазону.

]:

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from skimage import data
```

- В первой ячейке данного блока происходит импорт библиотек. Все они уже нам знакомы)

- Загружаем изображение с астронавтом из модуля data, импортированного из библиотеки skimage в первой ячейке

```
# 1. Загружаем изображение (RGB)
img_rgb = data.astronaut()
```

Далее пишем строчку кода для перевода изображения из пространства RGB, в котором оно загрузилось из data в HSV. В этом цветовом пространстве применять маски удобнее т.к. легче задать более точный диапазон. Результат (новое изображение, хранимое в виде массива Numpy) складываем в переменную img_hsv

```
img_hsv = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)
```

- В OpenCV значение Hue (тон) меняется от 0 до 180. Это «круг», где:
- 0 — начало круга,
- 180 — конец круга,

но 180 и 0 — это одно и то же место (красный цвет).

- Красный цвет сидит ровно на стыке: часть его оттенков попадает в начало шкалы (0–10), а часть — в конец (170–180). Если задать только один диапазон [0, 10] или [170, 180] , мы потеряем половину красных оттенков.
- Поэтому красный всегда описывается двумя парами границ, которые потом объединяются.
- Почему S ≥ 100 и V ≥ 100:
- S ≥ 100 — отсекаем бледные, сероватые оттенки. Если насыщенность ниже 100, цвет уже не красный, а розовато-серый.
- V ≥ 100 — отсекаем слишком тёмные пиксели. Если яркость ниже 100, цвет почти чёрный, и мы не хотим, чтобы такие пиксели попадали в маску.
- Эти пороги можно подстраивать: например, если объект освещён плохо, стоит понизить V до 50. Если наоборот — изображение пересвечено, можно поднять нижнюю границу S.

[3]
0
сек.

```
# 3. Задаём диапазоны красного цвета в HSV
# Красный в HSV занимает два интервала из-за цикличности Hue
lower_red1 = np.array([0, 100, 100])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 100, 100])
upper_red2 = np.array([180, 255, 255])
```

Этот блок **готовит «фильтр» для поиска красного цвета** на изображении. Он ещё ничего не ищет — он только **описывает, что считать красным.**

- Как мы знаем маска задается диапазоном значений цвета: верхняя и нижняя граница. В программе это реализуется через функцию inRange библиотеки cv2. В качестве аргументов мы передаем название переменной, которая хранит нужное изображение в виде массива Numpy, массив, задающий нижнюю границу, массив, задающий верхнюю границу – 3 параметра.

- Как мы выяснили ранее красный состоит из двух диапазонов. Нам нужно объединить их, спрятав в одну маску. Т.к. нам нужно чтобы пиксель, предполагаемый красным, попадал в ОДИН ИЗ этих двух диапазонов, то можем привести аналогию операции логического или. После маски пиксель будет белым если попал ИЛИ в первый диапазон ИЛИ во второй. Такое объединение масок возможно через функцию bitwise_or, куда передаем обе маски. Итоговую маску складываем в переменную mask. Там хранится черно-белое изображение после применения маски.

- Далее последним шагом – применение маски к изображению. За это отвечает функция bitwise_and. В качестве аргументов передаются имя изображения, (дважды.. Потому что потому что так надо) и потом маска. Результат – цветное изображение, на котором видны только пиксели, попавшие в маску, остальное – чёрный фон. Сохраняем его в переменную result.

```
# 4. Создаём две маски и объединяем их
mask1 = cv2.inRange(img_hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(img_hsv, lower_red2, upper_red2)
mask = cv2.bitwise_or(mask1, mask2)

# 5. Применяем маску к исходному изображению
# (оставляем только те пиксели, где маска белая)
result = cv2.bitwise_and(img_rgb, img_rgb, mask=mask)
```

- Последняя ячейка **не требует дописывания кода** — она нужна только для визуализации. Мы выводим **три изображения рядом**, чтобы наглядно увидеть весь путь обработки:
- **Оригинал** — исходное изображение астронавта.
- **Маска** — чёрно-белое изображение, где белым выделены пиксели, попавшие в диапазон красного, а чёрным — все остальные.
- **Результат** — оригинал, к которому применена маска: видны только пиксели, соответствующие красному цвету, остальное — чёрный фон.
- Сравнивая три картинки, мы видим, что маска точно указывает, где на оригинале находится красный скафандр, а результат показывает, как выглядит изображение после «вырезания» по этой маске.

```
# 6. Визуализация
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Оригинал")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(mask, cmap='gray')
plt.title("Маска (красный цвет)")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(result)
plt.title("Только красные объекты")
plt.axis('off')

plt.tight_layout()
plt.show()
```




на таких котятах
и держится страна



Thanks for your attention!)